ARDUINO PARA PRINCIPIANTES

Guía Visual para tus Primeros Proyectos



APRENDE DESDE CERO

PROYECTOS PRÁCTICOS

EXPLICACIONES VISUALES

CREA, EXPERIMENTA Y DIVIÉRTETE

¡Bienvenido al Mundo de Arduino!

Estás a punto de comenzar una aventura increíble llena de creatividad, aprendizaje y proyectos que puedes llevar a la vida real.



¿Qué es Arduino?

Arduino es una plataforma electrónica de código abierto que te permite crear proyectos interactivos de manera fácil y divertida.



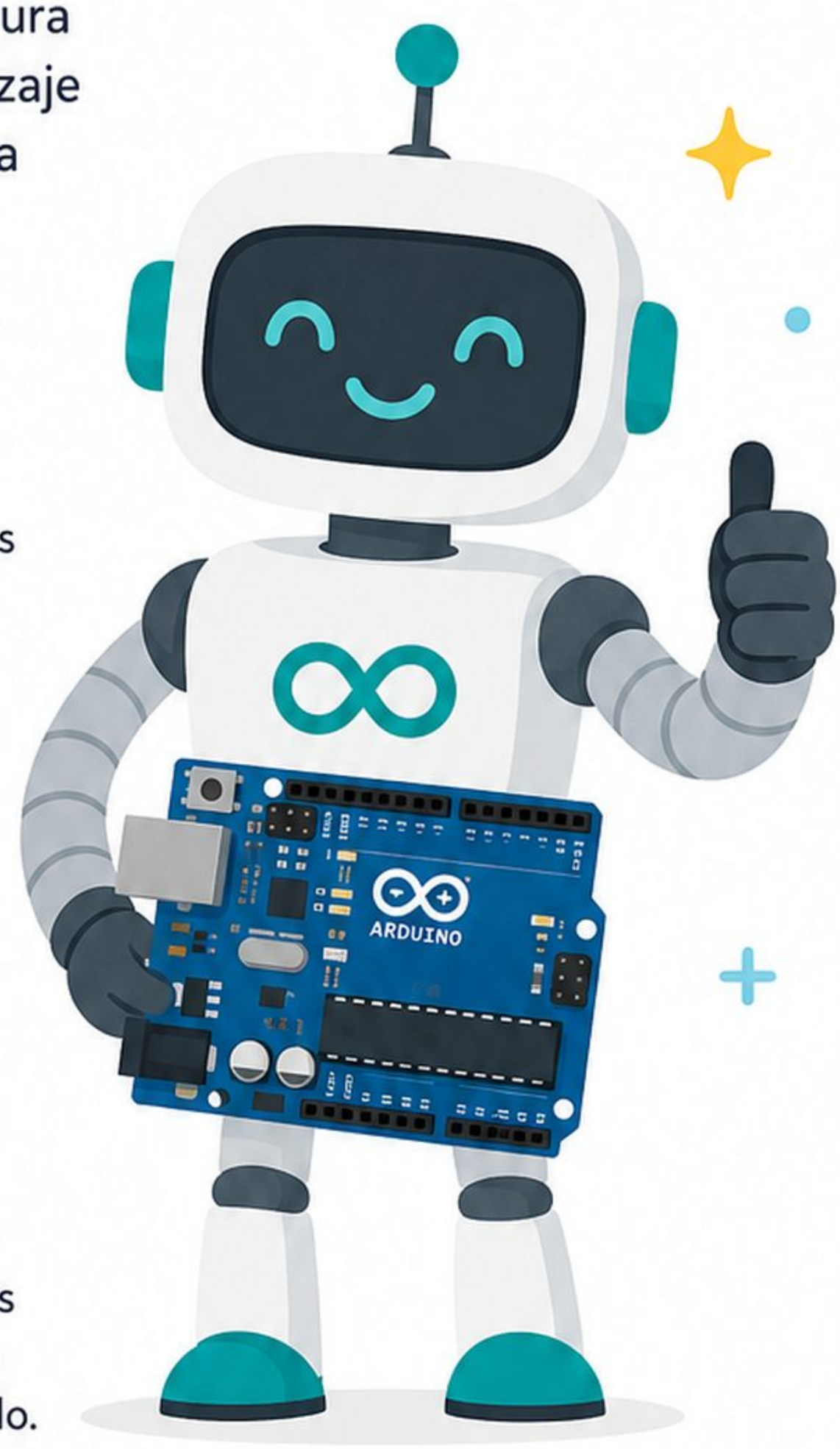
¿Qué podrás hacer?

Desde encender un LED hasta construir robots, estaciones meteorológicas y mucho más. ¡Tú pones el límite!



¿Por qué es especial?

Porque aprenderás haciendo, cometerás errores, descubrirás soluciones y te convertirás en un creador de tu propio mundo.



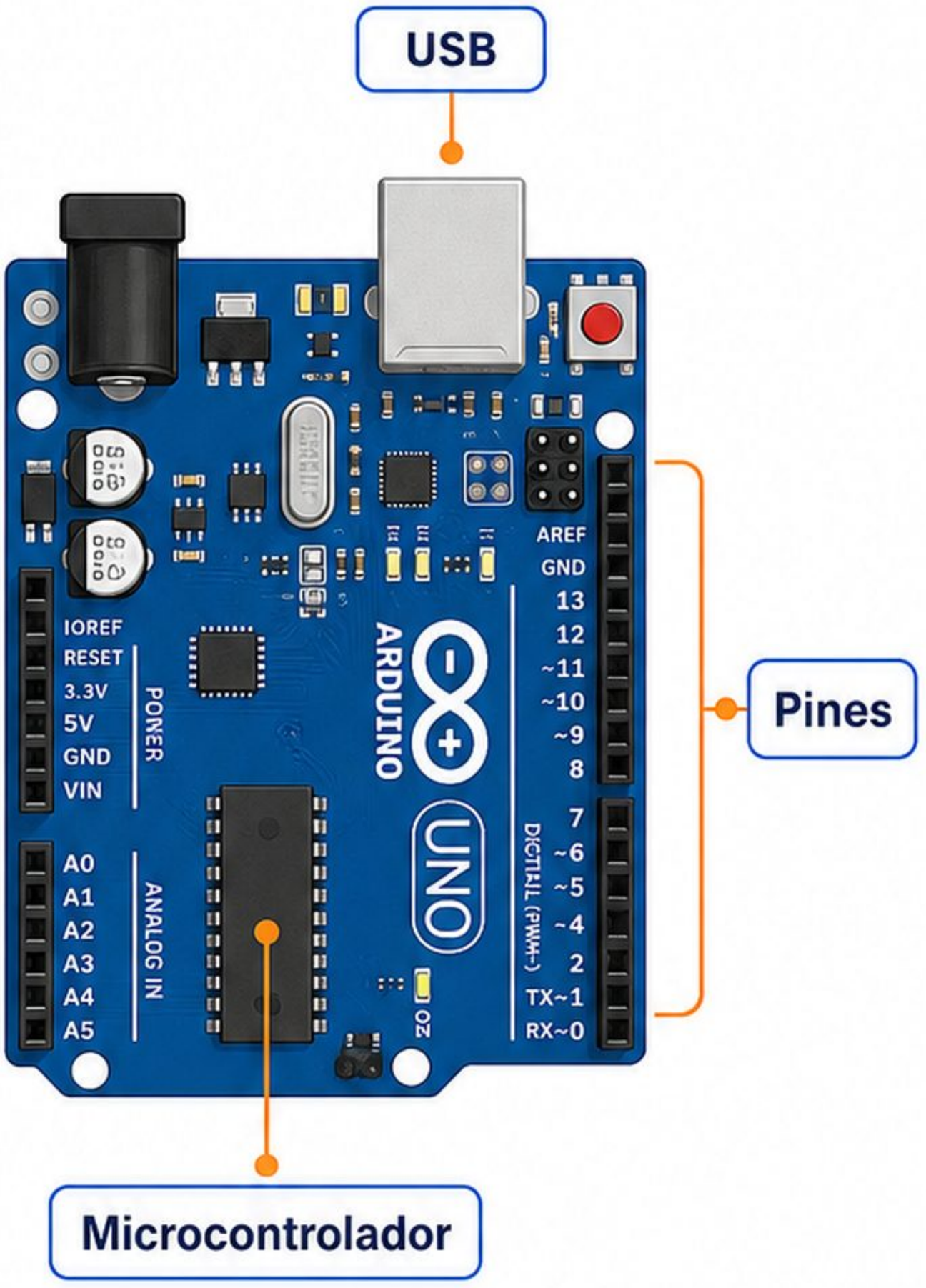


No necesitas ser un experto para empezar.
Solo curiosidad, ganas de aprender y ¡empecemos!

¡Vamos a crear cosas increíbles juntos! ♥

¿QUÉ ES ARDUINO?

Arduino es una plataforma de hardware libre que permite crear proyectos electrónicos interactivos. Incluye una placa programable y un entorno de desarrollo (IDE) para escribir código.



CREA

Desarrolla proyectos electrónicos interactivos de manera sencilla.

PERSONALIZA

Hardware y software abiertos para adaptar y ampliar según tus necesidades.

APRENDE

Ideal para estudiantes, aficionados y profesionales.

Ideal para:

APRENDER

Entiende electrónica y programación de forma práctica.

PROTOTIPAR

Crea y prueba ideas rápidamente.

AUTOMATIZAR

Automatiza procesos y mejora tu entorno con tecnología.

HISTORIA Y COMUNIDAD

1

2005

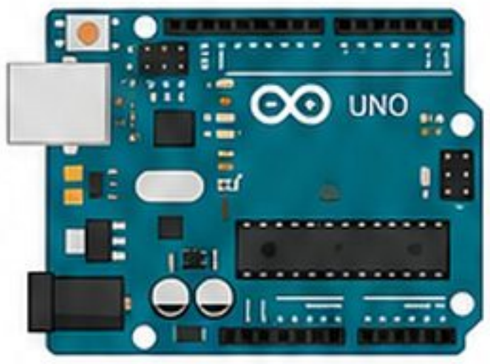


NACE EN ITALIA

Un grupo de diseñadores e ingenieros crea Arduino en Ivrea, Italia.

2

2006



PRIMERA PLACA UNO

Se lanza la placa Arduino Uno, fácil de usar y accesible para todos.

3

2012



UN MILLÓN DE PLACAS

Se alcanzan las un millón de placas vendidas en todo el mundo.

4

HOY



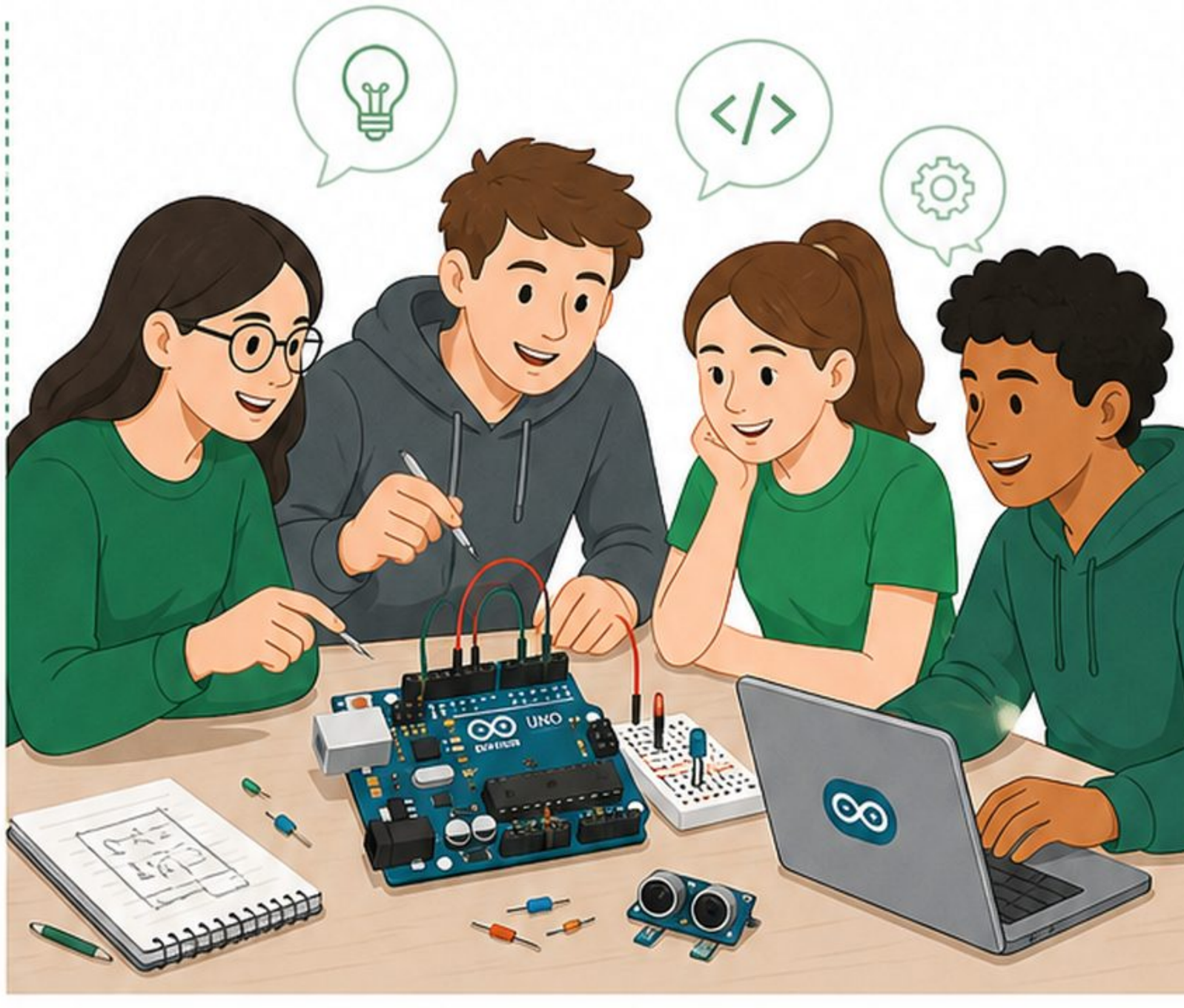
COMUNIDAD GLOBAL

Millones de personas aprenden, enseñan y crean proyectos con Arduino en todo el mundo.

Arduino es open-source:

miles de tutoriales y proyectos compartidos por la comunidad.



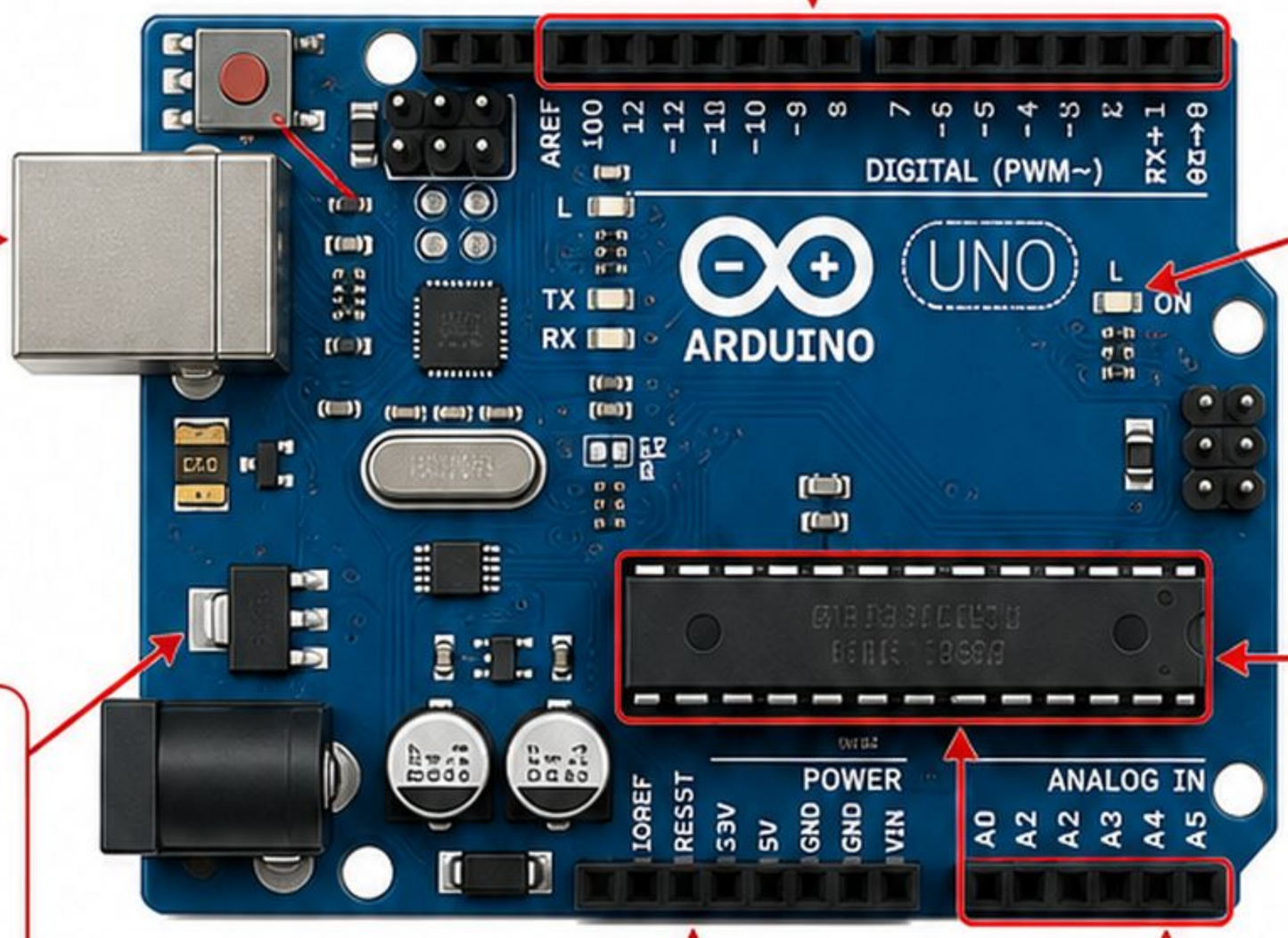


HECHO POR PERSONAS, **PARA PERSONAS.**

COMPONENTES DE LA PLACA ARDUINO UNO

Pines digitales
Permiten la conexión de dispositivos digitales.

USB
Permite la comunicación con la computadora y la alimentación de la placa.



LED integrado
Indica el estado de la placa. Conectado al pin 13.

Botón reset
Reinicia el programa que se está ejecutando.

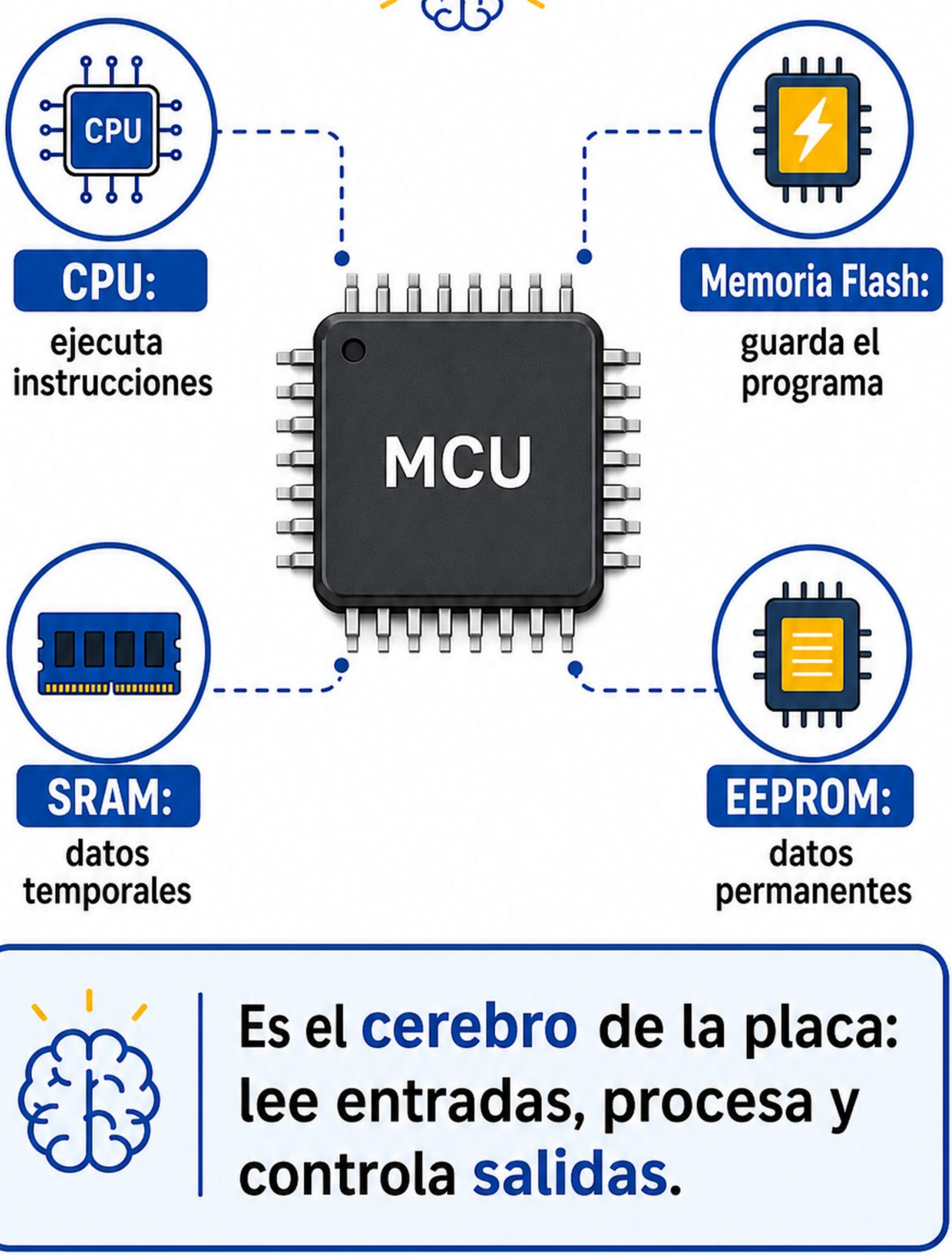
Regulador de voltaje
Estabiliza el voltaje para proteger la placa y sus componentes.

Microcontrolador
Es el cerebro de la placa; ejecuta el programa.

Pines analógicos
Permiten la conexión de sensores analógicos.

Componente	Función
USB	comunicación y alimentación
Microcontrolador	procesa el programa
Pines	conexión con sensores y actuadores

MICROCONTROLADOR:
EL CEREBRO DE LA PLACA



CPU:
ejecuta instrucciones

Memoria Flash:
guarda el programa

SRAM:
datos temporales

EEPROM:
datos permanentes

Es el **cerebro** de la placa: lee entradas, procesa y controla **salidas**.

PINES
DIGITALES Y ANALÓGICOS

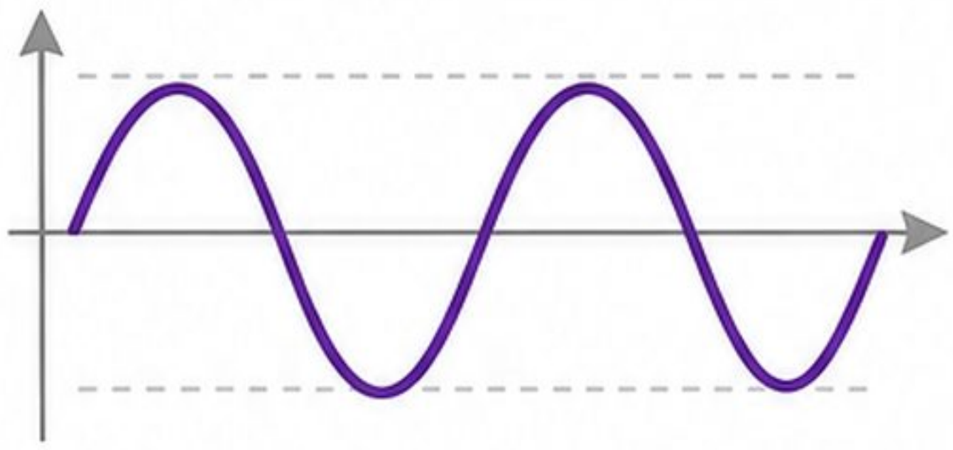
DIGITALES



Solo dos estados:
HIGH (5V) o **LOW** (0V).

Se usan para LEDs,
botones, relés.

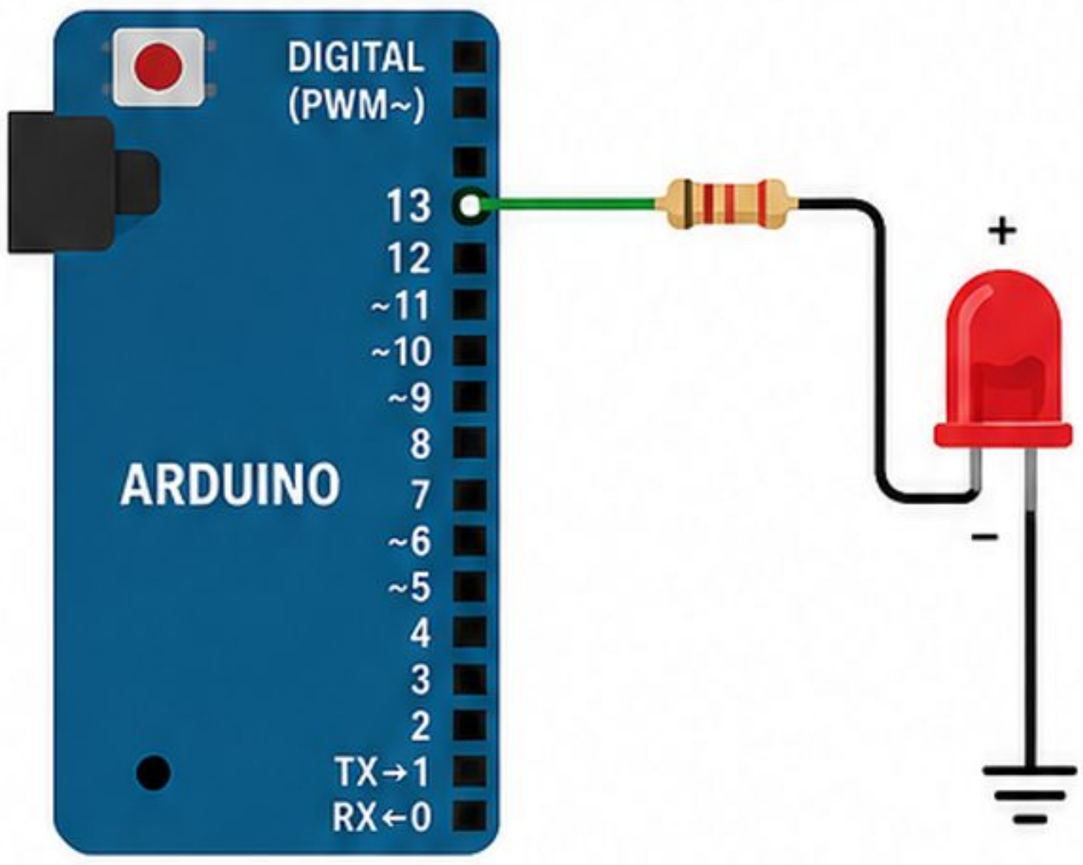
ANALÓGICOS



Leen valores continuos
de **0** a **1023**.

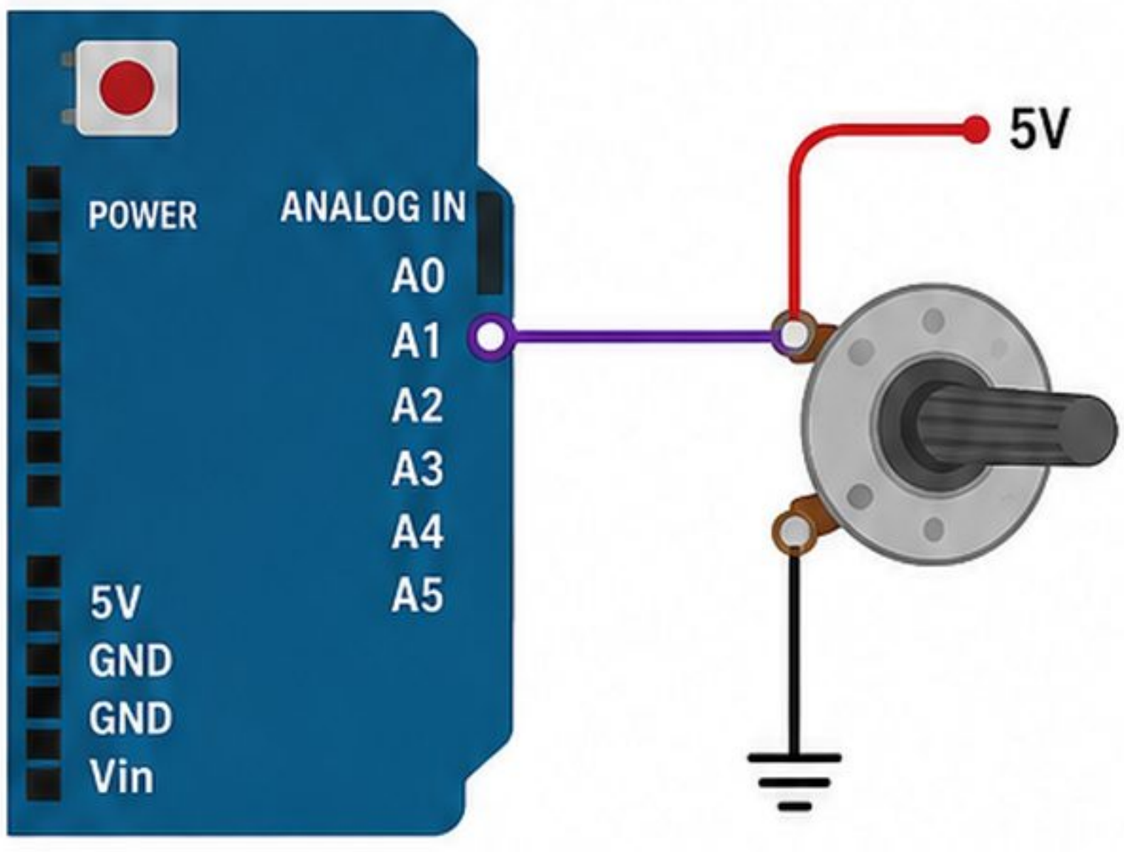
Se usan para sensores
de luz, temperatura,
potenciómetros.

EJEMPLO: LED EN PIN DIGITAL



Pin digital (13) → Resistencia → LED → GND

EJEMPLO: POTENCIÓMETRO EN PIN ANALÓGICO



5V → Potenciómetro → GND
Salida (cursor) → Pin analógico (A0)

RECUERDA: Los pines digitales tienen dos estados (**0** o **1**).
Los pines analógicos leen valores continuos (**0** a **1023**).

ALIMENTACIÓN Y COMUNICACIÓN

ALIMENTACIÓN

USB:
5V desde el ordenador

BATERÍA:
6-20V en el conector jack

PIN 5V:
salida regulada

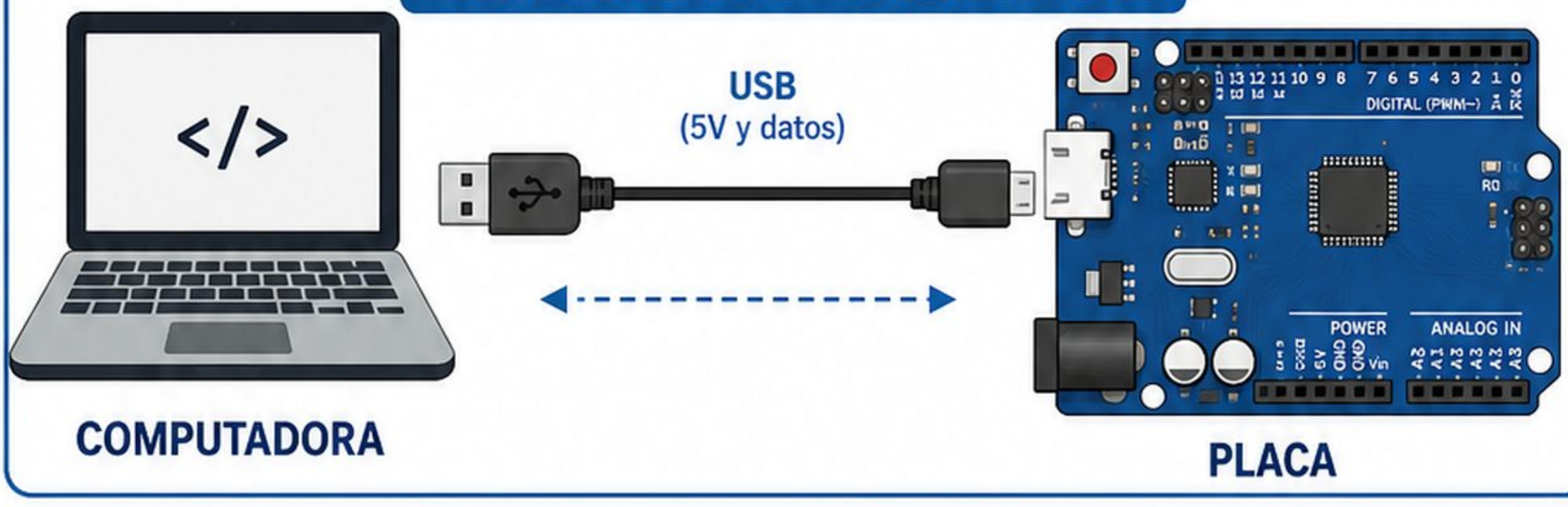
COMUNICACIÓN

USB:
carga y comunicación serial

UART:
comunicación con otros dispositivos

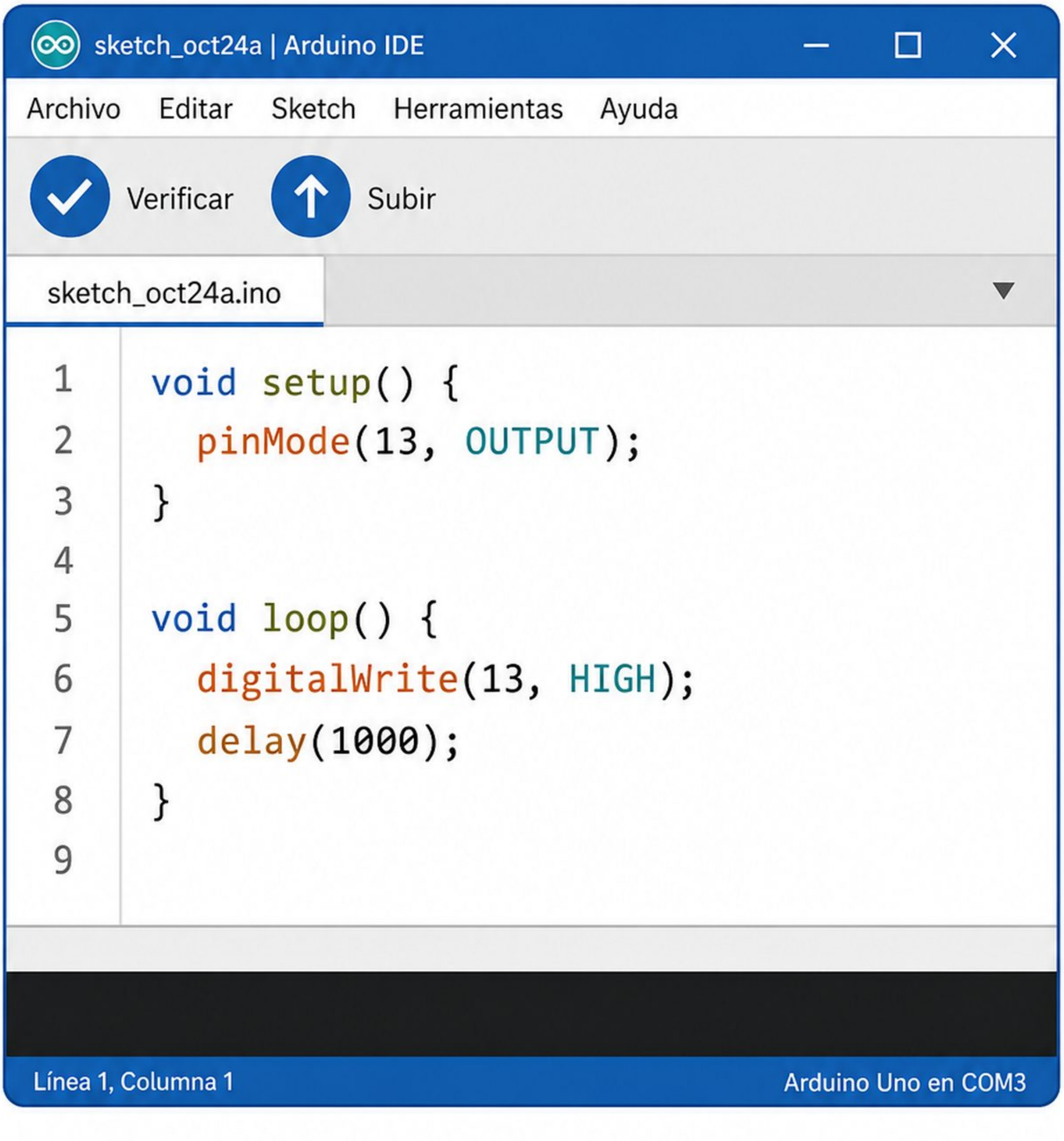
I²C Y SPI:
protocolos para sensores

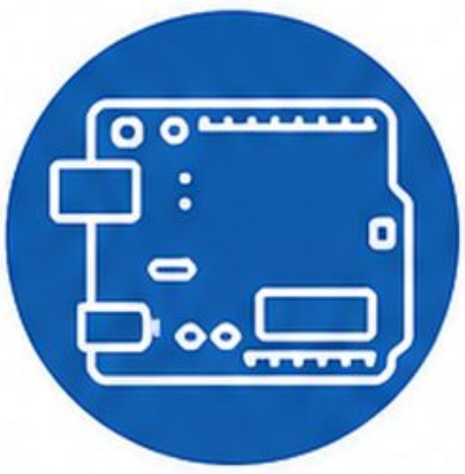
CONEXIÓN PLACA – COMPUTADORA



TIP: Alimenta la placa con una fuente adecuada y utiliza el USB para programar y comunicarte con ella.

ENTORNO DE DESARROLLO (IDE)





Escribe tu programa, **verifica** errores y **súbelo** a la placa.

PRIMER CIRCUITO:
LED PARPADEANTE

MATERIALES



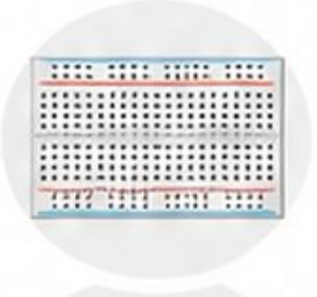
Placa Arduino Uno



LED



Resistencia 220Ω

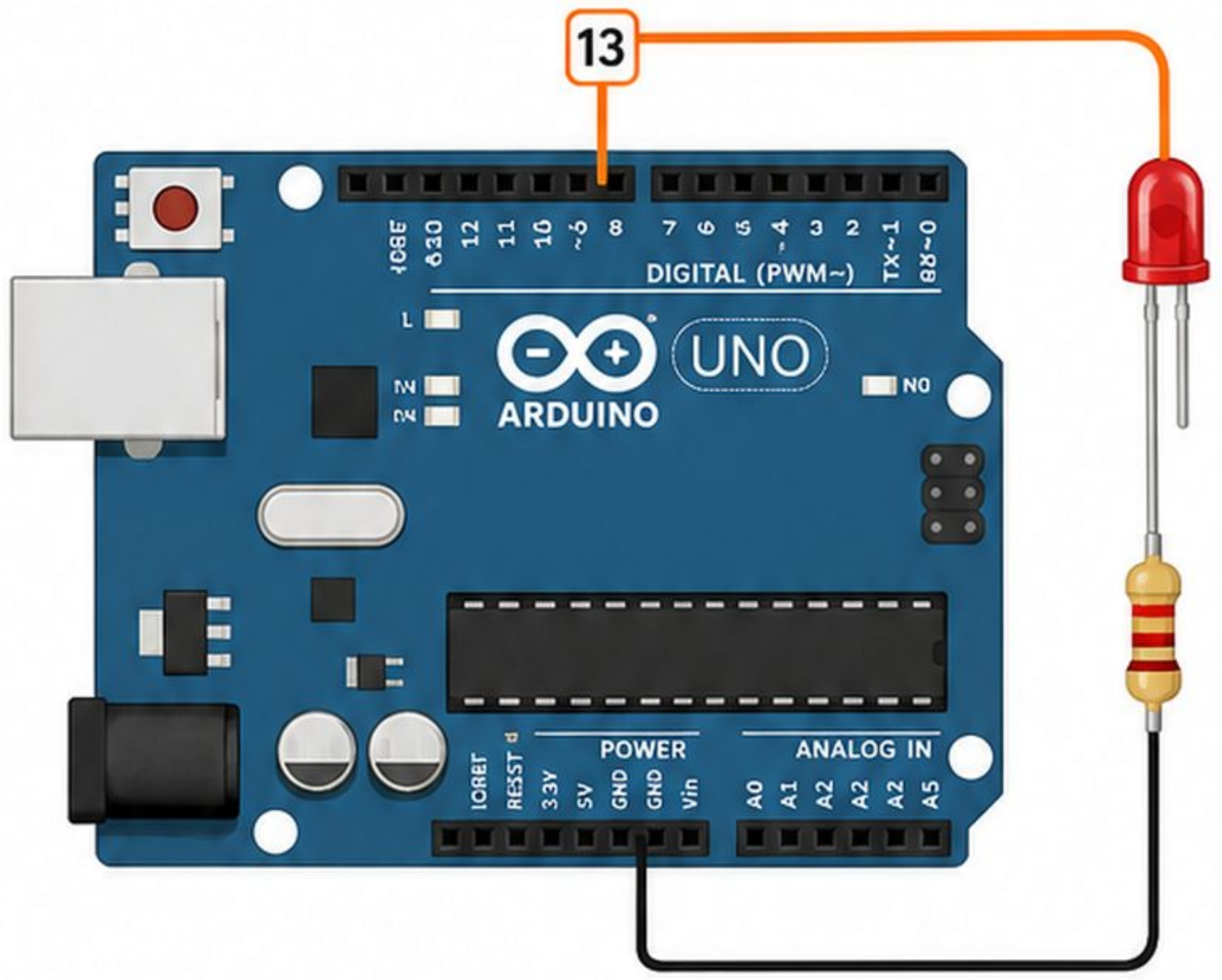


Protoboard



Cables jumper

DIAGRAMA DEL CIRCUITO



1. Conecta el LED al pin 13
2. Conecta la resistencia a GND
3. Sube el código

CÓDIGO

```
void setup() {
   pinMode(13, OUTPUT);
}

void loop() {
   digitalWrite(13, HIGH);
   delay(1000);
   digitalWrite(13, LOW);
   delay(1000);
}
```

EXPLICACIÓN DEL CÓDIGO

```
1   void setup() {
2     pinMode(13, OUTPUT);
3   }
4
5   void loop() {
6     digitalWrite(13, HIGH);
7     delay(1000);
8     digitalWrite(13, LOW);
9     delay(1000);
10  }
11
12
13
14
15
```

Se ejecuta una vez al inicio

Configura el pin 13 como salida

Se repite infinitamente

Enciende el LED

Espera 1 segundo

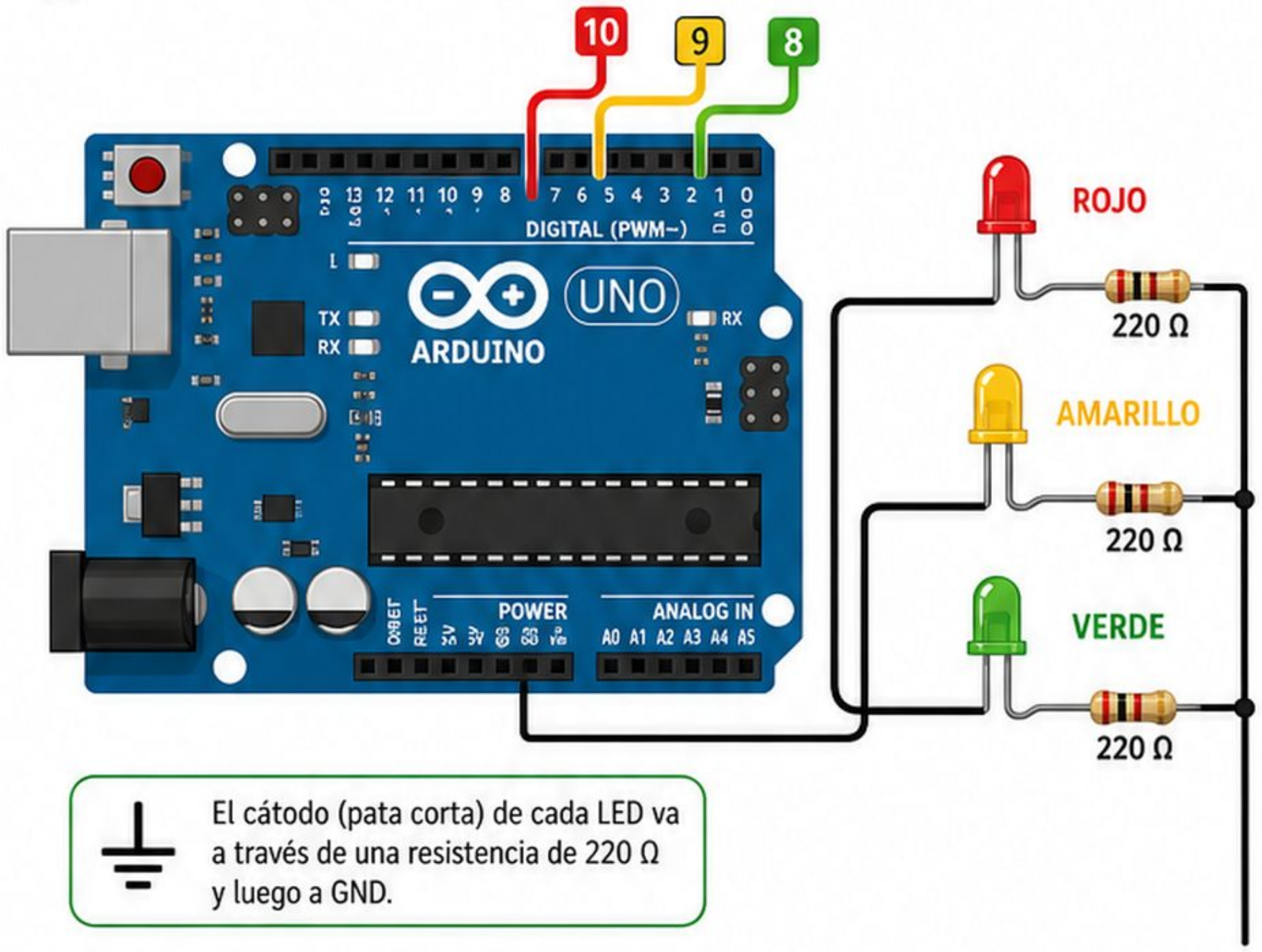
Apaga el LED

Espera 1 segundo

El código es la receta que la placa sigue paso a paso.

Proyecto: Semáforo con LEDs

DIAGRAMA DE CONEXIÓN



El cátodo (pata corta) de cada LED va a través de una resistencia de 220 Ω y luego a GND.

MATERIALES



3 LEDs

- 1 Rojo
- 1 Amarillo
- 1 Verde



3 Resistencias

- 220 ohmios (220 Ω)



Cables

- Macho–Macho (varios)

CÓDIGO DE EJEMPLO: SEMÁFORO

```
// Semáforo con LEDs
// Pines: 10 = Rojo, 9 = Amarillo, 8 = Verde

const int ROJO = 10;
const int AMARILLO = 9;
const int VERDE = 8;

void setup() {
  pinMode(ROJO, OUTPUT);
  pinMode(AMARILLO, OUTPUT);
  pinMode(VERDE, OUTPUT);
}

void loop() {
  // 1. Verde encendido (avanzar)
  digitalWrite(VERDE, HIGH);
  delay(5000);
  digitalWrite(VERDE, LOW);
  delay(500);

  // 2. Amarillo encendido (precaución)
  digitalWrite(AMARILLO, HIGH);
  delay(2000);
  digitalWrite(AMARILLO, LOW);
  delay(500);

  // 3. Rojo encendido (detener)
  digitalWrite(ROJO, HIGH);
  delay(5000);
  digitalWrite(ROJO, LOW);
  delay(500);
}
```



1. VERDE
Avanzar
5 segundos



2. AMARILLO
Precaución
2 segundos



3. ROJO
Detener
5 segundos

TIP:
Puedes modificar los tiempos en las funciones `delay()` para ajustar la duración de cada luz.

¡Construye tu propio semáforo y aprende programación con Arduino!

Simulación del Semáforo

El semáforo cambia de estado en una secuencia cíclica controlada por tiempos definidos.

SECUENCIA DE ESTADOS (LÍNEA DE TIEMPO)

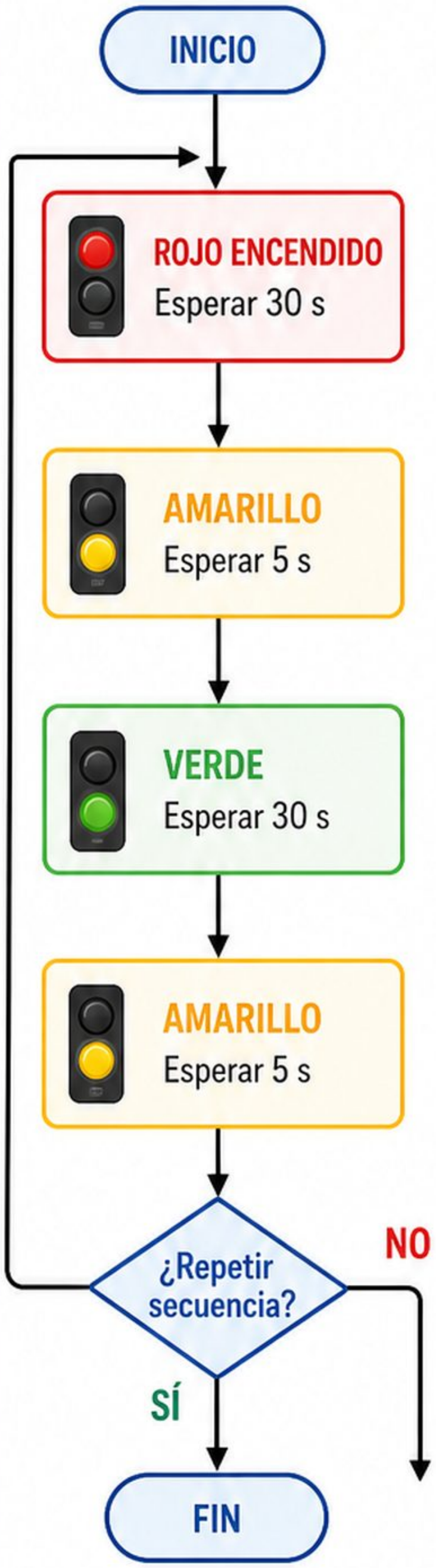


DETALLE DE LOS ESTADOS

Rojo encendido	30 s	Detener el paso. Precaución máxima.
Amarillo	5 s	Atención. El cambio es inminente.
Verde	30 s	Avanzar con precaución. Vía libre.
Amarillo	5 s	Atención. El cambio es inminente.
Repetición (cíclica)	–	La secuencia se repite indefinidamente.

Nota: Los tiempos pueden ajustarse según la necesidad de la simulación.

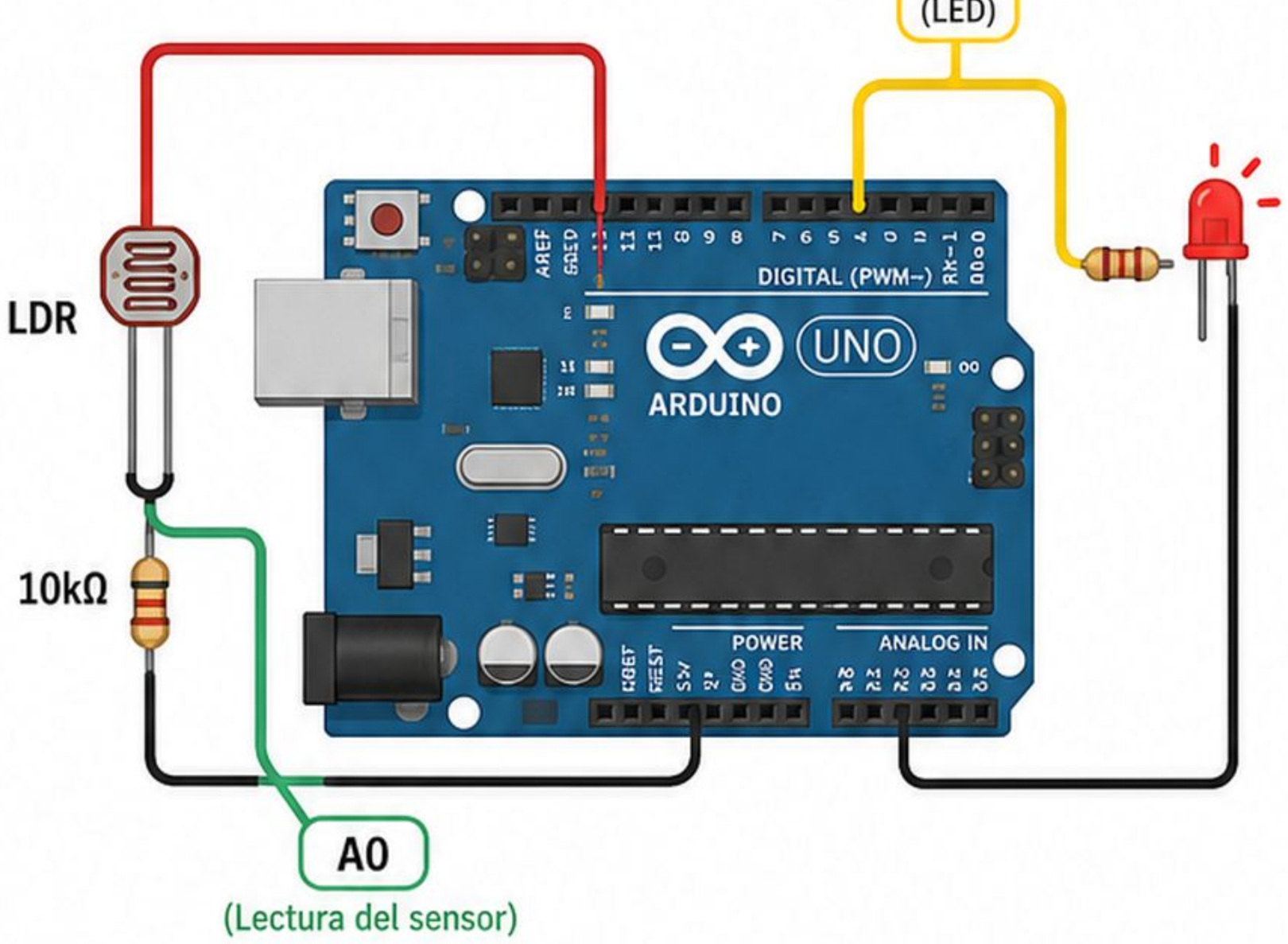
DIAGRAMA DE FLUJO (LÓGICA DEL CÓDIGO)



TIP: Usa funciones de espera (delay) o temporizadores para controlar los tiempos de cada estado.

Proyecto: Sensor de Luz

DIAGRAMA DE CONEXIÓN



MATERIALES

- LDR
- Resistencia de 10k ohmios
- LED
- Cables (macho-macho)

CÓDIGO DE EJEMPLO

```
const int pinLDR = A0;     // Pin analógico donde se conecta el LDR
const int pinLED = 9;      // Pin digital donde se conecta el LED

void setup() {
  pinMode(pinLED, OUTPUT);   // Configura el pin del LED como salida
  Serial.begin(9600);        // Inicia comunicación serial (opcional)
}

void loop() {
  int valorLuz = analogRead(pinLDR); // Lee el valor del sensor (0 a 1023)
  Serial.print("Luz: ");
  Serial.println(valorLuz);          // Muestra el valor en el Monitor Serie

  if (valorLuz < 400) {              // Si la luz es baja (umbral ajustable)
    digitalWrite(pinLED, HIGH);      // Enciende el LED
  } else {
    digitalWrite(pinLED, LOW);       // Apaga el LED
  }
  delay(200);                        // Pequeña pausa para estabilidad
}
```

¿CÓMO FUNCIONA?

Con **MUCHA LUZ** el sensor lee un valor **ALTO** y el LED permanece **APAGADO**.

Con **POCA LUZ** el sensor lee un valor **BAJO** y el LED se **ENCIENDE**.

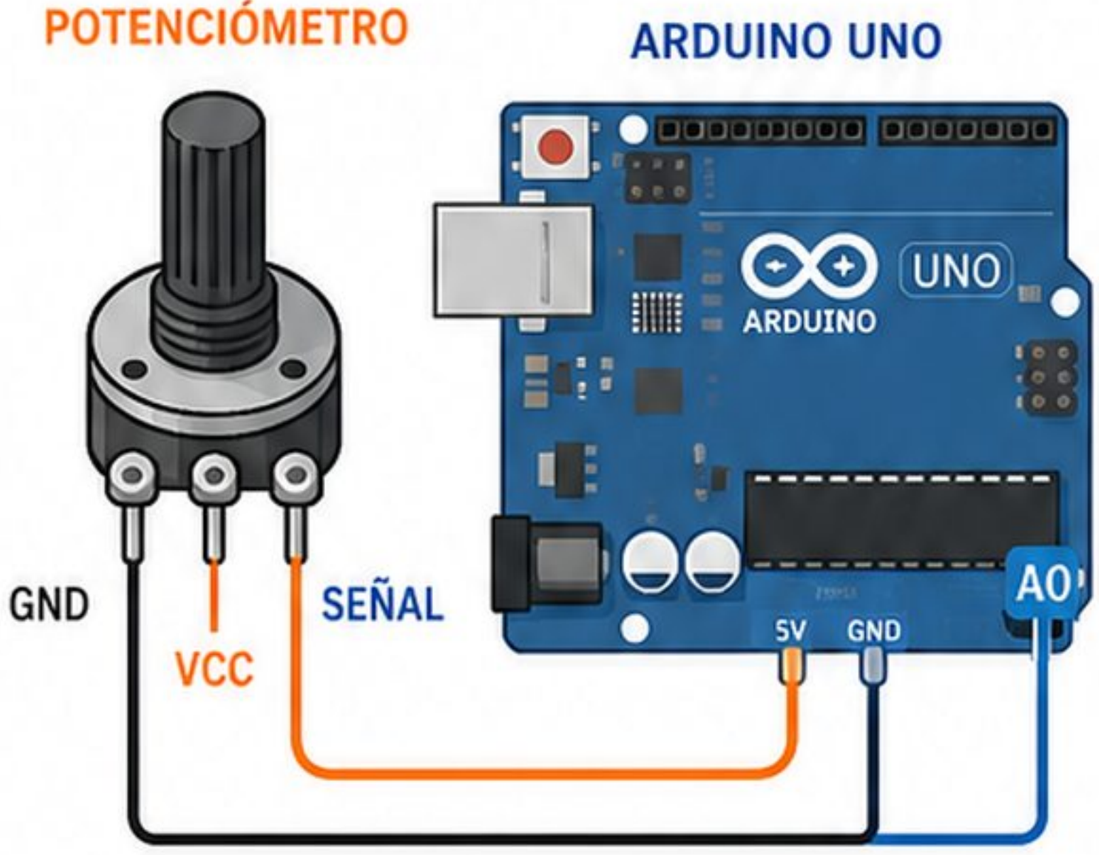
TIPS

- El umbral (valorLuz < 400) puedes ajustarlo según tu entorno.
- Abre el Monitor Serie (9600 baudios) para ver los valores del sensor.
- Cubre o ilumina el LDR para probar el funcionamiento.

Lectura de Sensores Analógicos

1. CONEXIÓN

Conectamos un potenciómetro a la entrada analógica A0 del Arduino Uno.

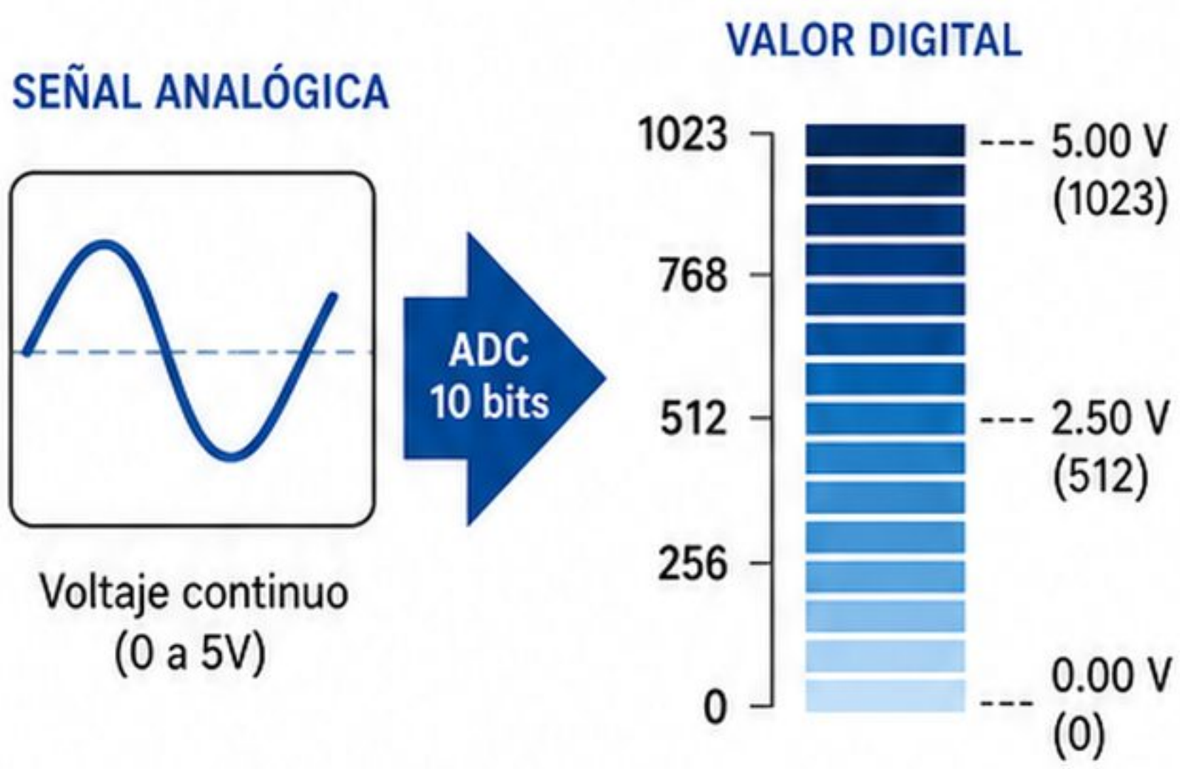


- GND → Tierra
- VCC → 5V
- SEÑAL (cursor) → A0

2. ¿CÓMO FUNCIONA?

Arduino Uno tiene un Conversor Analógico-Digital (ADC) de **10 bits**.

Convierte el voltaje analógico de entrada (0 a 5V) en un número entero entre **0 y 1023**.



IMPORTANTE

Más voltaje de entrada = mayor número digital de salida (0 a 1023).

3. CÓDIGO DE EJEMPLO

```
1  void setup() {
2     Serial.begin(9600);
3  }
4
5  void loop() {
6     int valor = analogRead(A0);  // Lee el valor analógico (0-1023)
7     Serial.println(valor);     // Muestra el valor en el Monitor Serie
8     delay(100);                // Pequeña pausa para estabilidad
9  }
10
11
```

MONITOR SERIE

```
512
513
514
515
514
513
512
...
```

Valores entre **0 y 1023**

ANALÓGICO
Señal continua (potenciómetro)

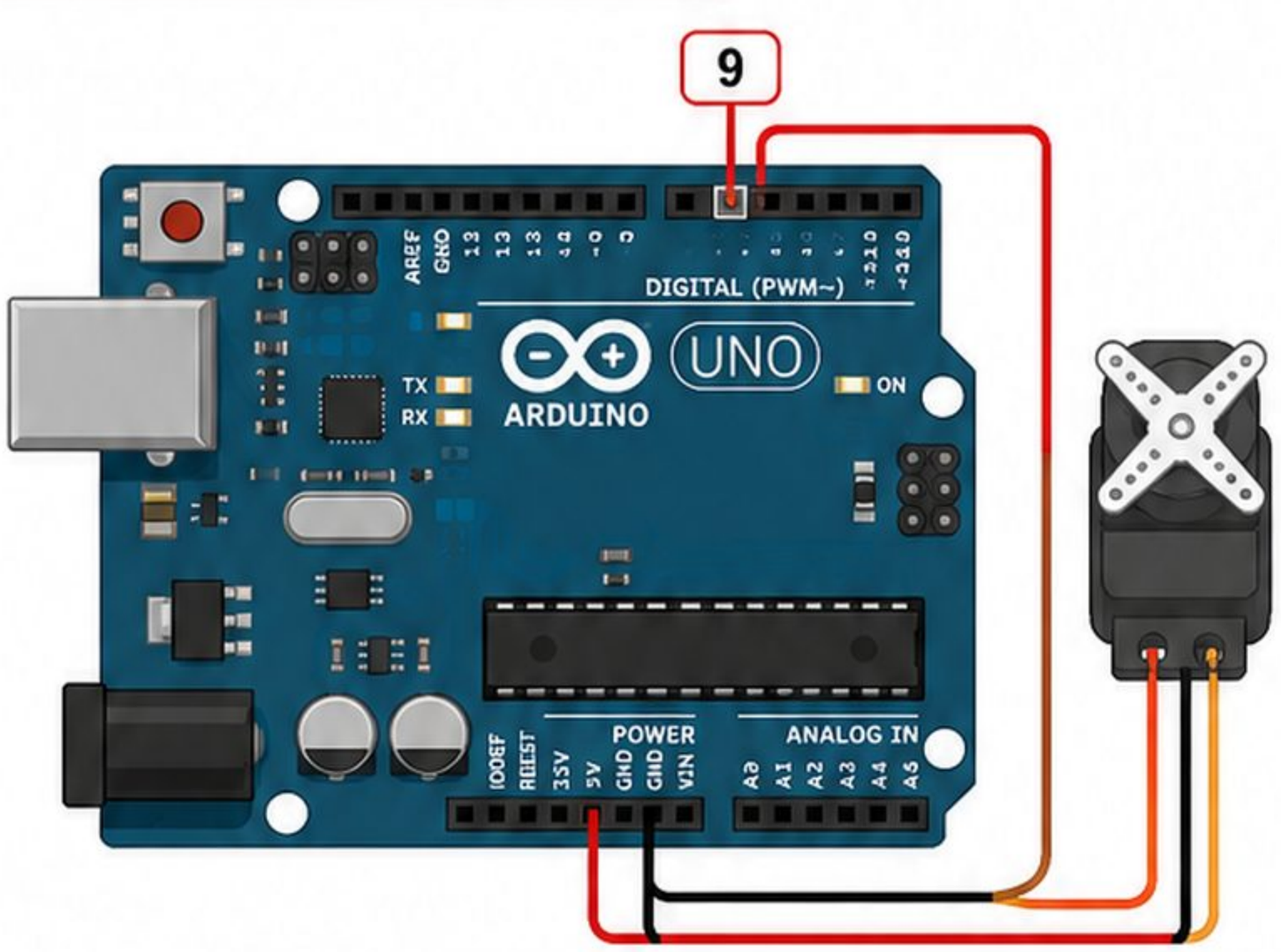
CONVERSIÓN
El ADC de 10 bits convierte el voltaje a un número entre 0 y 1023.

DIGITAL
Número entero que Arduino puede procesar y mostrar.

TIP: Puedes usar cualquier sensor analógico (LDR, LM35, micrófono, etc.) conectado a **A0–A5.**

Proyecto:
Control de Servomotor

1. DIAGRAMA DE CONEXIÓN

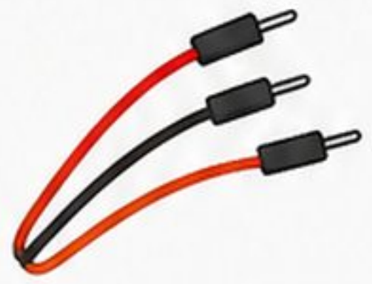


- Señal (NARANJA) → Pin digital 9
- VCC (ROJO) → 5V
- GND (MARRÓN/NEGRO) → GND

2. MATERIALES



- Servomotor (ej. SG90)



- Cables (macho-hembra)

¿CÓMO FUNCIONA?

El pin 9 envía una señal PWM al servomotor para indicar la posición. El servo se mueve de 0° a 180°.

3. CÓDIGO DE EJEMPLO

```
#include <Servo.n>

Servo miServo;
const int pinServo = 9;

void setup() {
  miServo.attach(pinServo); // Conecta el servo al pin 9
}

void loop() {
  // Mover de 0 a 180 grados
  for (int angulo = 0; angulo <= 180; angulo++) {
    miServo.write(angulo);  // Enviar posición al servo
    delay(15);              // Pequeña pausa
  }
}
```



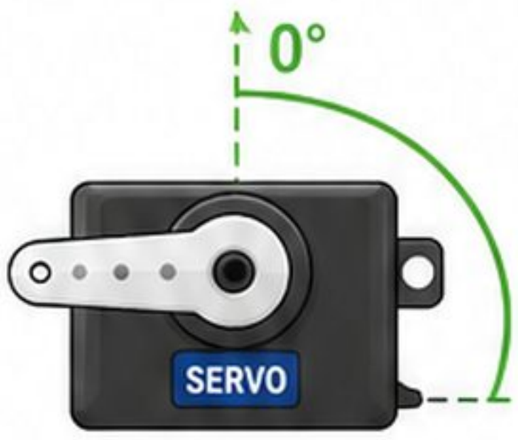
¡EXPERIMENTA! Modifica la velocidad cambiando el valor de delay(15) y prueba mover el servo a posiciones específicas.

Movimiento del Servo

EL SERVO EN DIFERENTES ÁNGULOS

0°

Posición inicial (mínimo)



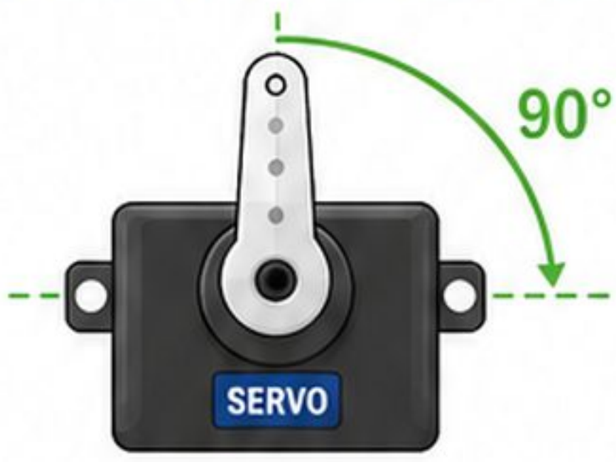
45°

Gira 45° en sentido horario



90°

Posición central



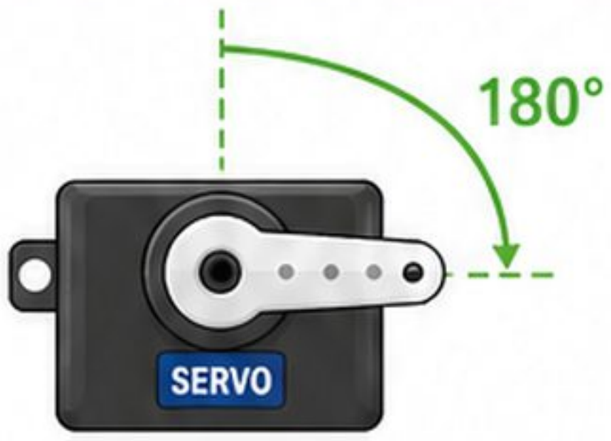
135°

Gira 135° en sentido horario



180°

Posición final (máximo)



¿CÓMO USAR LA LIBRERÍA DEL SERVOMOTOR?

En Arduino, la librería Servo permite controlar fácilmente el movimiento del servomotor.

1. Incluye la librería:

```
#include <Servo.h>
```

2. Crea un objeto Servo:

```
Servo miServo;
```

3. Asocia el servo a un pin:

```
miServo.attach(9);
```

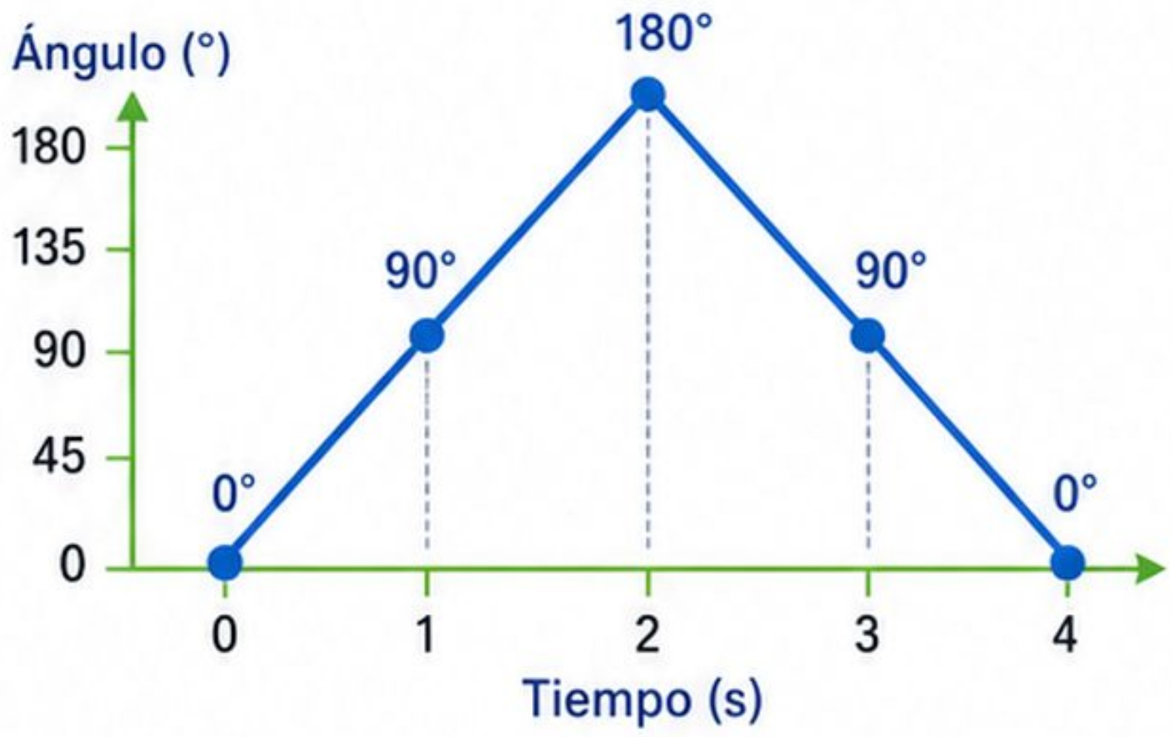
4. Escribe el ángulo (0° a 180°):

```
miServo.write(90);
```

El comando write ( ) coloca el servo en el ángulo indicado.

POSICIÓN DEL SERVO A LO LARGO DEL TIEMPO

Ejemplo: el servo se mueve entre diferentes ángulos en el tiempo.

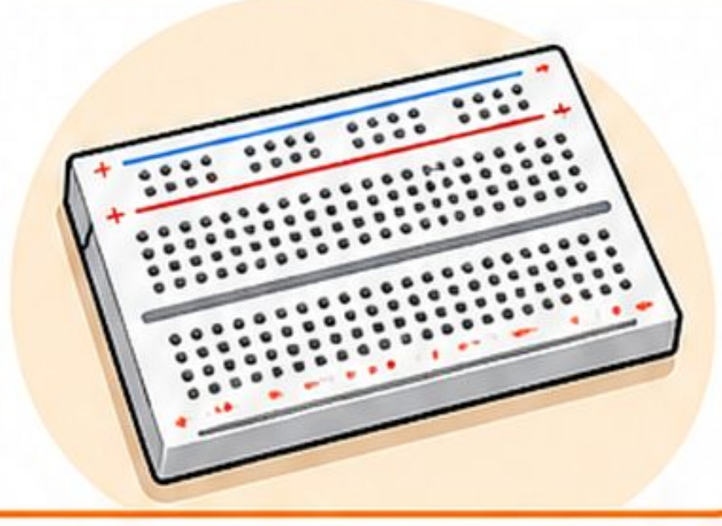


RECUERDA

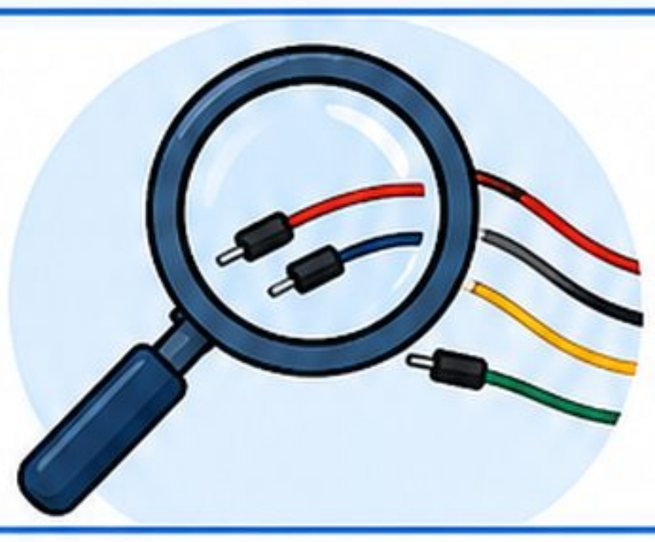
El rango típico de un servomotor es de 0° a 180°.
Los valores fuera de este rango pueden causar un comportamiento inesperado.

Consejos para tus Proyectos

1 Usa una protoboard para prototipar



2 Revisa las conexiones antes de alimentar



3 Empieza con proyectos simples



4 Documenta tus circuitos



5 Experimenta y diviértete





Cada proyecto es un paso hacia tu MEJOR VERSIÓN.



Siguientes Pasos

Sigue aprendiendo, explorando y creando.
El mundo de **Arduino** tiene infinitas posibilidades.

Recursos recomendados



Documentación oficial de Arduino

Guías, referencias y ejemplos para aprender desde lo básico hasta lo avanzado.



Kits de iniciación

Todo lo que necesitas para comenzar a crear proyectos increíbles.



Foros de la comunidad

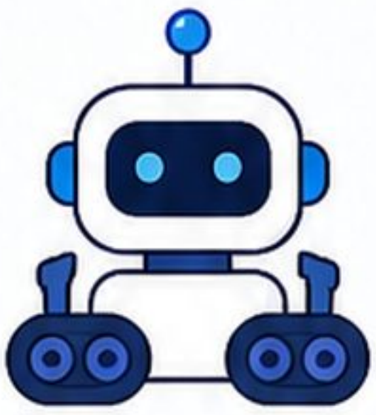
Comparte dudas, ideas y soluciones con personas de todo el mundo.

Ideas de proyectos



Automatización del hogar

Controla luces, sensores y dispositivos para hacer tu hogar más inteligente.



Robot seguidor de línea

Construye un robot que siga líneas y evita obstáculos.



Riego automático

Crea un sistema que riega tus plantas cuando más lo necesitan.



¡El límite es tu imaginación!